```
from google.colab import drive
import os, glob, subprocess, glob, numpy as np, matplotlib.pyplot as
plt
from scipy import signal
import soundfile as sf
from sklearn.decomposition import NMF
```

```
from google.colab import drive
import os
drive.mount('/content/drive')
sound_dir = '/content/drive/MyDrive/Sound/120s'
os.chdir(sound_dir)
print(f"Current working directory: {os.getcwd()}")
```

```
Mounted at /content/drive
Current working directory: /content/drive/MyDrive/Sound/120s
```

```
m4a_files = glob.glob('*.m4a')
print("Found .m4a files:", m4a_files)
```

```
Found .m4a files: ['120s-50.m4a']
```

```
def convert_m4a_to_wav(m4a_file, wav_file):
  res = subprocess.run(['ffmpeg', '-y', '-i', m4a_file, wav_file],
capture_output=True, text=True)
  print("ffmpeg returncode:", res.returncode)
  if res.returncode != 0:
      print("ffmpeg stderr:", res.stderr[:1000])
      raise RuntimeError("ffmpeg failed to convert the file")
  print("Converted file exists:", os.path.exists(wav_file))
  return res

def read_wav(wav_file):
  y, sr = sf.read(wav_file)
  if y.ndim > 1:
      y = np.mean(y, axis=1)
  y = y.astype(np.float32)
  duration = len(y) / sr
  return y, sr, duration

def STFT_shape(y, sr):  # Short-Time Fourier Transform
  n_fft = 2048
  hop_length = 512
  #
  f, t, Zxx = signal.stft(y, fs=sr, nperseg=n_fft, noverlap=n_fft-
hop_length, boundary=None)
  S = np.abs(Zxx)
  return S, f, t, Zxx, n_fft, hop_length

def separate_with_NMF(S, n_max_components): # Non-negative Matrix
Factorization (NMF) for separation
```

```
  S_max = S.max() if S.max() > 0 else 1.0
  S_norm = S / S_max + 1e-10
  recons_scores = {}
  for n_comp in range(2,n_max_components):
      model = NMF(n_components=n_comp, init='nndsvda', solver='mu',
beta_loss='kullback-leibler', max_iter=500, random_state=0)
      W = model.fit_transform(S_norm)
      H = model.components_
      S_approx = np.dot(W,H)
      err = np.linalg.norm(S_norm - S_approx, ord='fro')
      fit = 1 - (err / np.linalg.norm(S_norm, ord='fro'))
      recons_scores[n_comp] = fit
      print(f"n={n_comp} fit={fit:.4f}")
  return recons_scores, S_max, S_norm

def choose_best_n(recons_scores):
  prev = None
  best_n = 6
  for n, fit in sorted(recons_scores.items()):
      if prev is not None:
          if fit - prev < 0.02:
              best_n = n
              break
      prev = fit
  return best_n

def NMF_recompute(S, best_n):
  model = NMF(n_components=best_n, init='nndsvda', solver='mu',
beta_loss='kullback-leibler', max_iter=500, random_state=0)
  W = model.fit_transform(S_norm); H = model.components_
  components_S = []
  for k in range(best_n):
      comp = np.outer(W[:,k], H[k,:]) * S_max
      components_S.append(comp)
  components_S = np.array(components_S)
  return components_S

def mask_reconstruct(components_S, Zxx):
  eps = 1e-10
  sum_S = np.sum(components_S, axis=0) + eps
  masks = components_S / sum_S
  reconstructed = []
  out_files = []
  for k in range(best_n):
      masked = masks[k] * Zxx
      _, y_comp = signal.istft(masked, fs=sr, nperseg=n_fft,
noverlap=n_fft-hop_length, input_onesided=True)
      y_comp = y_comp[:len(y)]
      reconstructed.append(y_comp)
      base, _ = os.path.splitext(m4a_file)
```

```
      outp = f"{base}_{k+1}.wav"
      sf.write(outp, y_comp, sr)
      out_files.append(outp)
  return masks, reconstructed, out_files

def plot_overlayed_waveforms(y, reconstructed, sr, duration):
  plt.figure(figsize=(12,6))
  t_axis = np.arange(len(y)) / sr
  plt.plot(t_axis, y / (np.max(np.abs(y)) + 1e-9), alpha=0.25,
linewidth=0.8, label='mixture (normalized)')
  for k, sig in enumerate(reconstructed):
      if np.max(np.abs(sig)) > 0:
          sign = sig / np.max(np.abs(sig))
      else:
          sign = sig
      # Create t_axis based on the length of the current signal
      t_axis_comp = np.arange(len(sign)) / sr
      plt.plot(t_axis_comp, sign, linewidth=1, label=f'component
{k+1}')
  plt.xlabel('Time (s)'); plt.ylabel('Amplitude (normalized)')
  plt.title('Overlayed waveforms: separated components (normalized)')
  plt.legend(loc='upper right', fontsize='small')
  plt.xlim(0, duration)
  plt.tight_layout()
  plt.show()
  plt.close()

def plot_individual_waveforms(y, reconstructed, sr, duration):
    n_signals = len(reconstructed)
    fig, axes = plt.subplots(n_signals + 1, 1, figsize=(12,
2*(n_signals+1)), sharex=True)

    # Plot the mixture on the first subplot
    t_axis = np.arange(len(y)) / sr
    axes[0].plot(t_axis, y / (np.max(np.abs(y)) + 1e-9), alpha=0.25,
linewidth=0.8, label='mixture (normalized)')
    axes[0].set_ylabel('Mixture')
    axes[0].legend(loc='upper right', fontsize='small')
    axes[0].set_title('Individual waveforms: separated components
(normalized)')

    # Plot each reconstructed component in its own subplot
    for k, sig in enumerate(reconstructed):
        if np.max(np.abs(sig)) > 0:
            sig_norm = sig / np.max(np.abs(sig))
        else:
            sig_norm = sig
        t_axis_comp = np.arange(len(sig_norm)) / sr
        axes[k+1].plot(t_axis_comp, sig_norm, linewidth=1,
label=f'component {k+1}')
```

```
        axes[k+1].set_ylabel(f'Comp {k+1}')
        axes[k+1].legend(loc='upper right', fontsize='small')

    axes[-1].set_xlabel('Time (s)')
    plt.xlim(0, duration)
    plt.tight_layout()
    plt.show()
    plt.close()
```

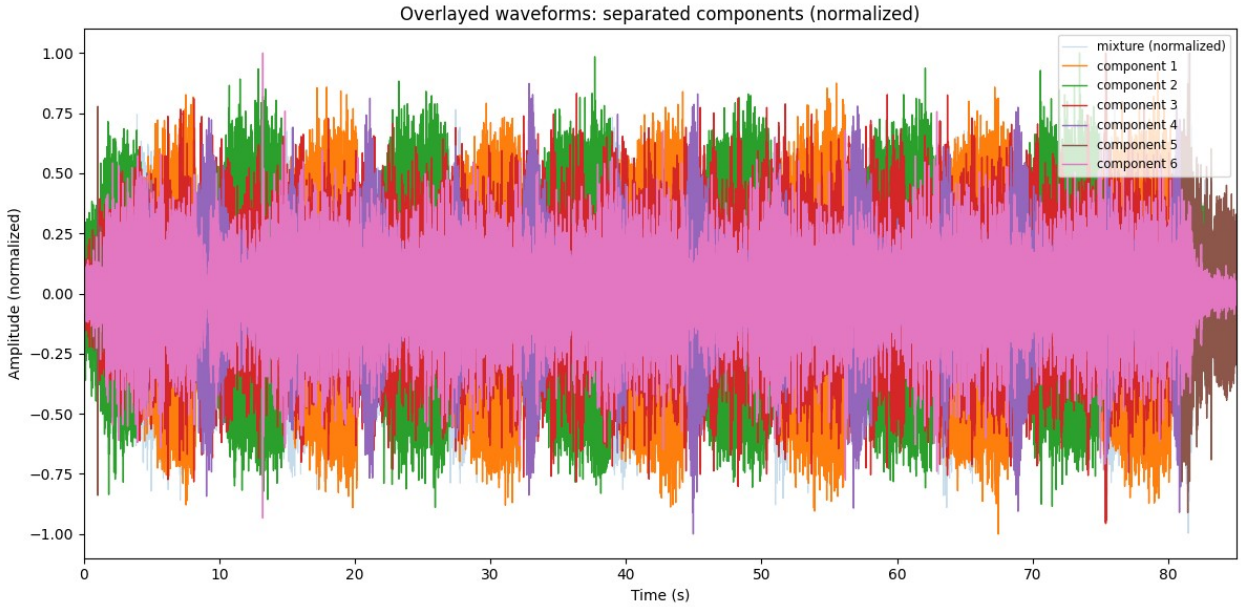
```
wav_files = []
for m4a_file in m4a_files:
    wav_file = os.path.splitext(m4a_file)[0] + '.wav'
    print(f"Converting {m4a_file} to {wav_file}...")
    res = convert_m4a_to_wav(m4a_file, wav_file)
    wav_files.append(wav_file)
    y, sr, duration = read_wav(wav_file)
    print(f"Loaded '{wav_file}' — duration: {duration:.2f}s, sample
rate: {sr} Hz, samples: {len(y)}")
    S, f, t, Zxx, n_fft, hop_length = STFT_shape(y, sr)
    print("STFT shape:", S.shape)
    n_max_components = 7
    recons_scores, S_max, S_norm = separate_with_NMF(S,
n_max_components)
    best_n = choose_best_n(recons_scores)
    print("Chosen components:", best_n)
    components_S = NMF_recompute(S, best_n)
    masks, reconstructed, out_files = mask_reconstruct(components_S,
Zxx)
    print("Saved outputs:", out_files)
    plot_overlayed_waveforms(y, reconstructed, sr, duration)
    plot_individual_waveforms(y, reconstructed, sr, duration)
print("Converted .wav files:", wav_files)
```

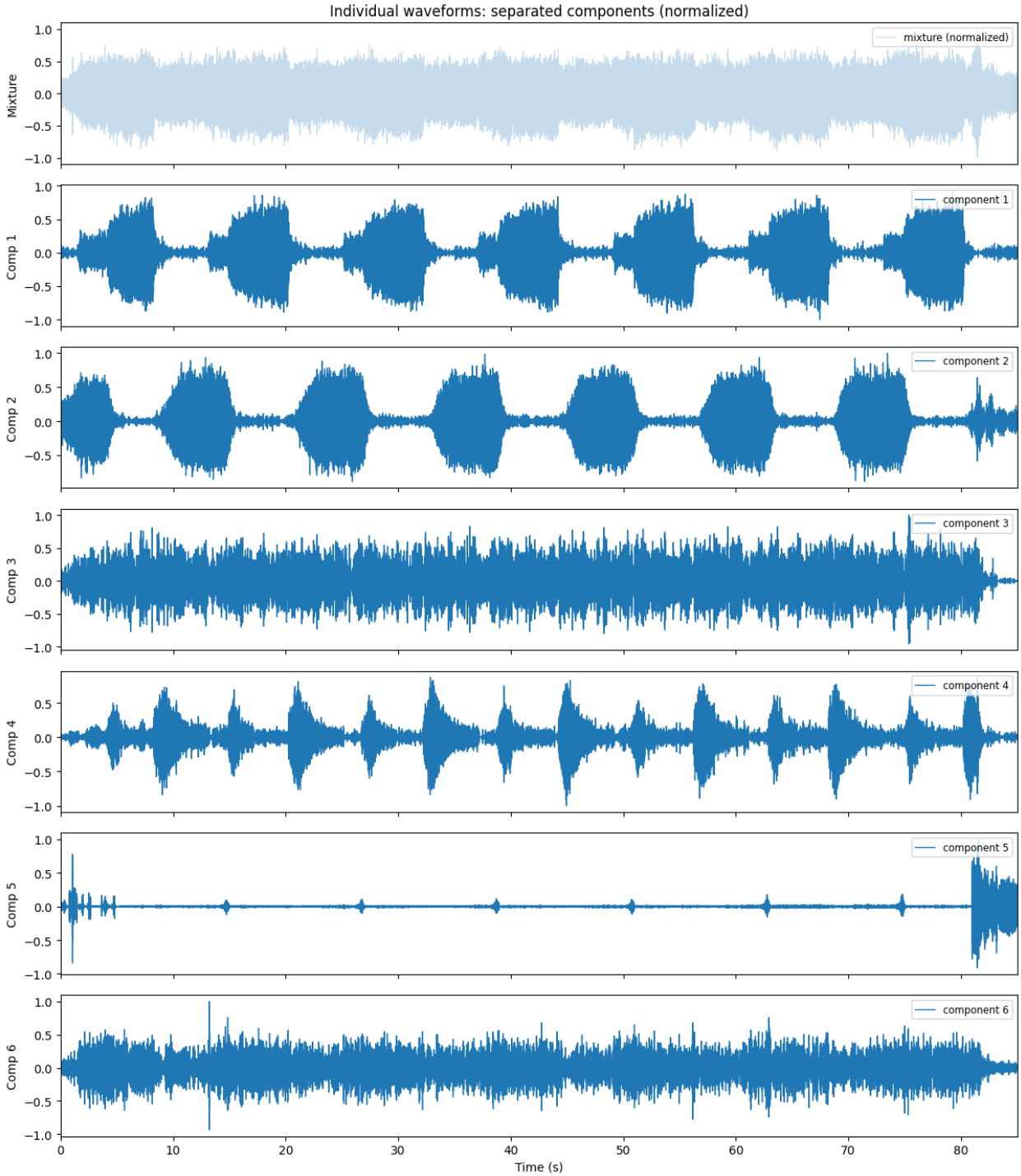
```
Converting 120s-50.m4a to 120s-50.wav...
ffmpeg returncode: 0
Converted file exists: True
Loaded '120s-50.wav' — duration: 85.03s, sample rate: 48000 Hz,
samples: 4081664
STFT shape: (1025, 7969)
n=2 fit=0.6289
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/decomposition/
_nmf.py:1742: ConvergenceWarning: Maximum number of iterations 500
reached. Increase it to improve convergence.
  warnings.warn(

n=3 fit=0.6495
n=4 fit=0.6799
```

```
n=5 fit=0.7033
n=6 fit=0.7195
Chosen components: 6
Saved outputs: ['120s-50_1.wav', '120s-50_2.wav', '120s-50_3.wav',
'120s-50_4.wav', '120s-50_5.wav', '120s-50_6.wav']
```






```
Converted .wav files: ['120s-50.wav']
```